54 Writing MCPs with FastMCP

Code

From decorator to deployable server in under an hour

54.1 Topic overview

The previous primer established what MCP is and why it matters. This one is about the other side of the protocol: writing an MCP server. The Python ecosystem has settled on **FastMCP** as the ergonomic way to do this — a decorator-driven framework that makes the simple case (one function, one tool) almost trivial, and that scales gracefully to servers with authentication, resources, prompts, and complex schemas.

The intuition to anchor on is that FastMCP is to MCP what FastAPI is to REST (Ramírez 2018). Both frameworks lean hard on Python's type hints to do work the developer would otherwise have to do by hand. In FastAPI, your type hints become the OpenAPI schema and the request validation logic. In FastMCP, your type hints become the JSON Schema that the LLM sees, the validator that runs on incoming arguments, and the documentation embedded in the tool description. Write the function correctly and almost everything else is taken care of.

This primer matters because writing MCP servers is now a baseline skill. When you encounter a tool or data source your agent does not already know about, your default move should be: write a small MCP server, ship it, point your host at it. The FastMCP path from idea to deployable server is short — measured in tens of lines of code, not hundreds — and the leverage is high: one server is reusable across every MCP-aware host you (or your team) will ever use. Once you have built a couple of them, you stop thinking of integrations as a special activity and start treating them as routine plumbing.

54.2 Background and theory

54.2.1 A brief history of FastMCP

When Anthropic announced MCP in November 2024, they released a reference Python SDK alongside the spec. That SDK was complete but low-level: you wrote handlers, registered them with a server object, and managed the protocol details directly. It worked, but it was the equivalent of writing a web app by handling raw HTTP request objects.

FastMCP emerged as a community project (initially developed by Jeremiah Lowin) within weeks of the protocol's announcement. Its pitch was the same as FastAPI's: use Python's type system and decorators to turn a normal function into a fully-described, validated, schema-emitting endpoint. The developer experience was so much better than the raw SDK that FastMCP rapidly became the default. By mid-2025, FastMCP 2.0 had been merged into the official Python SDK as the high-level interface, and "writing an MCP server in Python" essentially means "using FastMCP" (Lowin 2024).

The library is small enough that you can read most of it in an afternoon, which is part of its appeal. The decorator does not hide much; it inspects your function's signature and docstring, builds a JSON Schema, and registers the function as a handler. You can drop down to the lower-level SDK whenever you need to.

54.2.2 The minimal viable server

A complete, runnable MCP server in FastMCP looks like this:

```
from fastmcp import FastMCP

mcp = FastMCP("my-server")

@mcp.tool
```

```
def add(a: int, b: int) -> int:
    """Add two integers and return the sum."""
    return a + b

if __name__ == "__main__":
    mcp.run()
```

Six lines. No protocol code, no schema definitions, no JSON-RPC wiring. The decorator inspects `add`, extracts the parameter types (`int`, `int`), the return type (`int`), and the docstring ("Add two integers…"), and produces a tool registration with:

- Name: `add`
- Description: `"Add two integers and return the sum."`
- Input schema: `{"type": "object", "properties": {"a": {"type": "integer"}, "b": {"type": "integer"}}, "required": ["a", "b"]}`

The server, when launched (default transport: stdio), will respond to MCP protocol messages from any host. Plug it into Claude Desktop or your custom agent and it Just Works.

The structural relationship between the server object and its registered capabilities is small and uniform: a single `FastMCP` instance owns collections of tools, resources, and prompts, each created by a decorator wrapping a plain Python function.

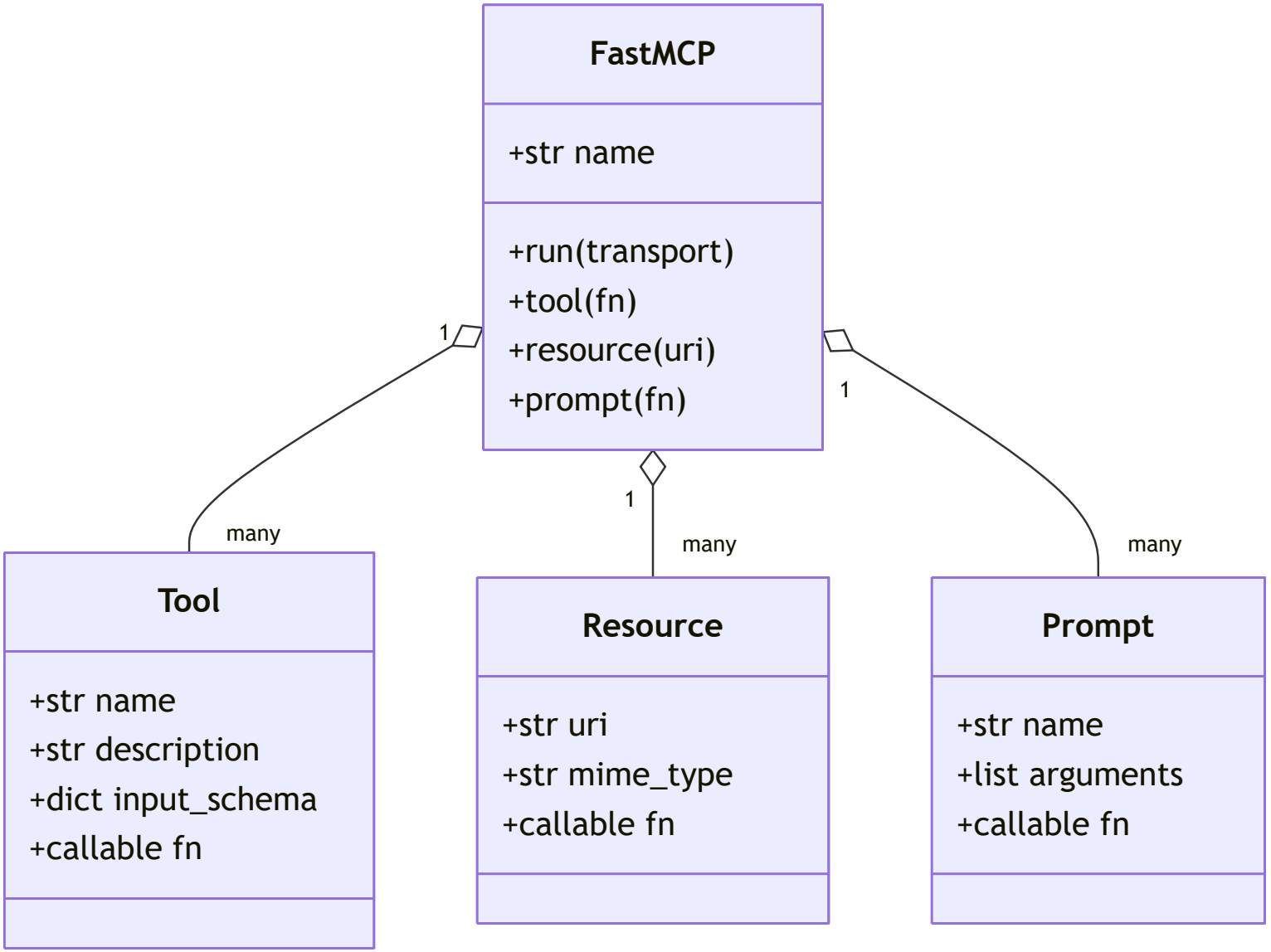
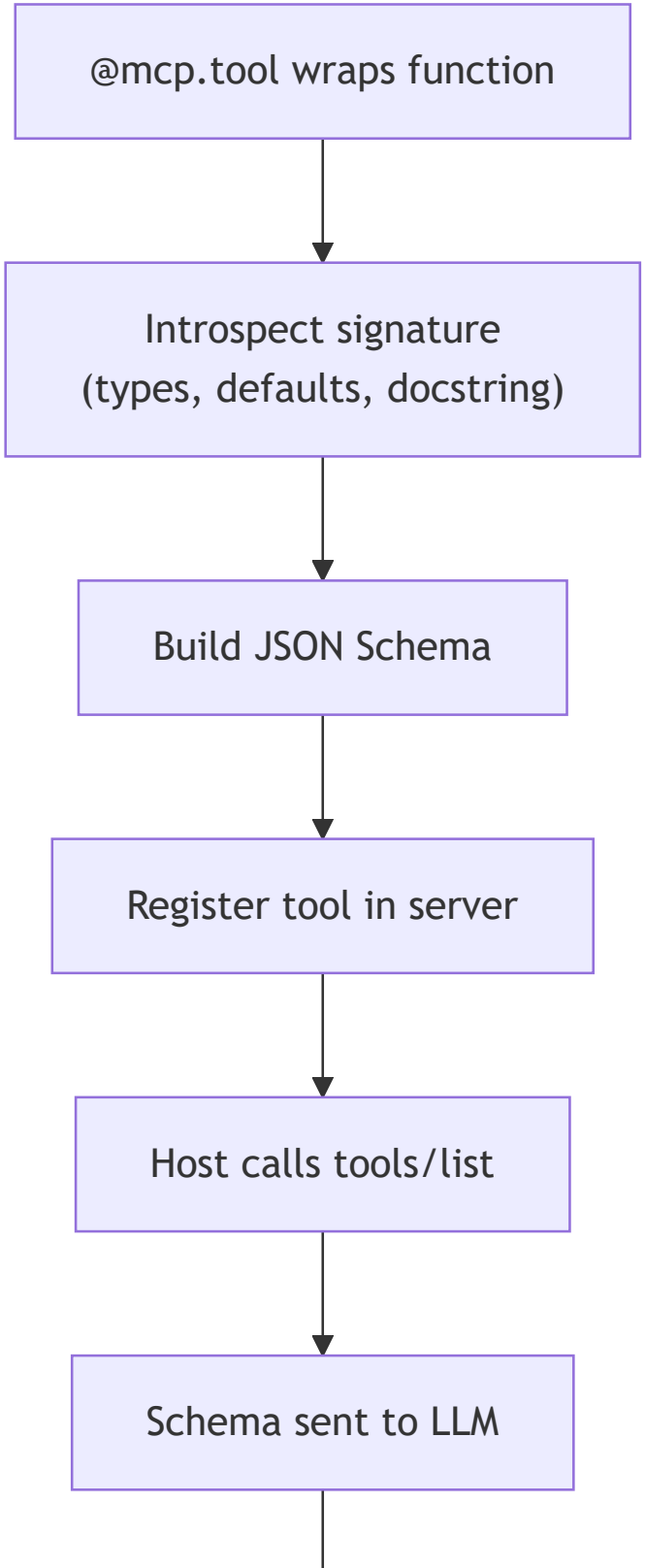


Figure 54.1: FastMCP object model: one server instance aggregates tools, resources, and prompts, each derived from a decorated Python function and its type hints.

54.2.3 Type hints become schemas

The deepest design choice in FastMCP is that **the function signature is the source of truth**. Every other piece of metadata — the input schema, the validation, the documentation — is derived from it.

The flow below traces what the `@mcp.tool` decorator does at registration time and what happens on each subsequent call.


@mcp.tool wraps function
Introspect signature
(types, defaults, docstring)
Build JSON Schema
Register tool in server
Host calls tools/list
Schema sent to LLM

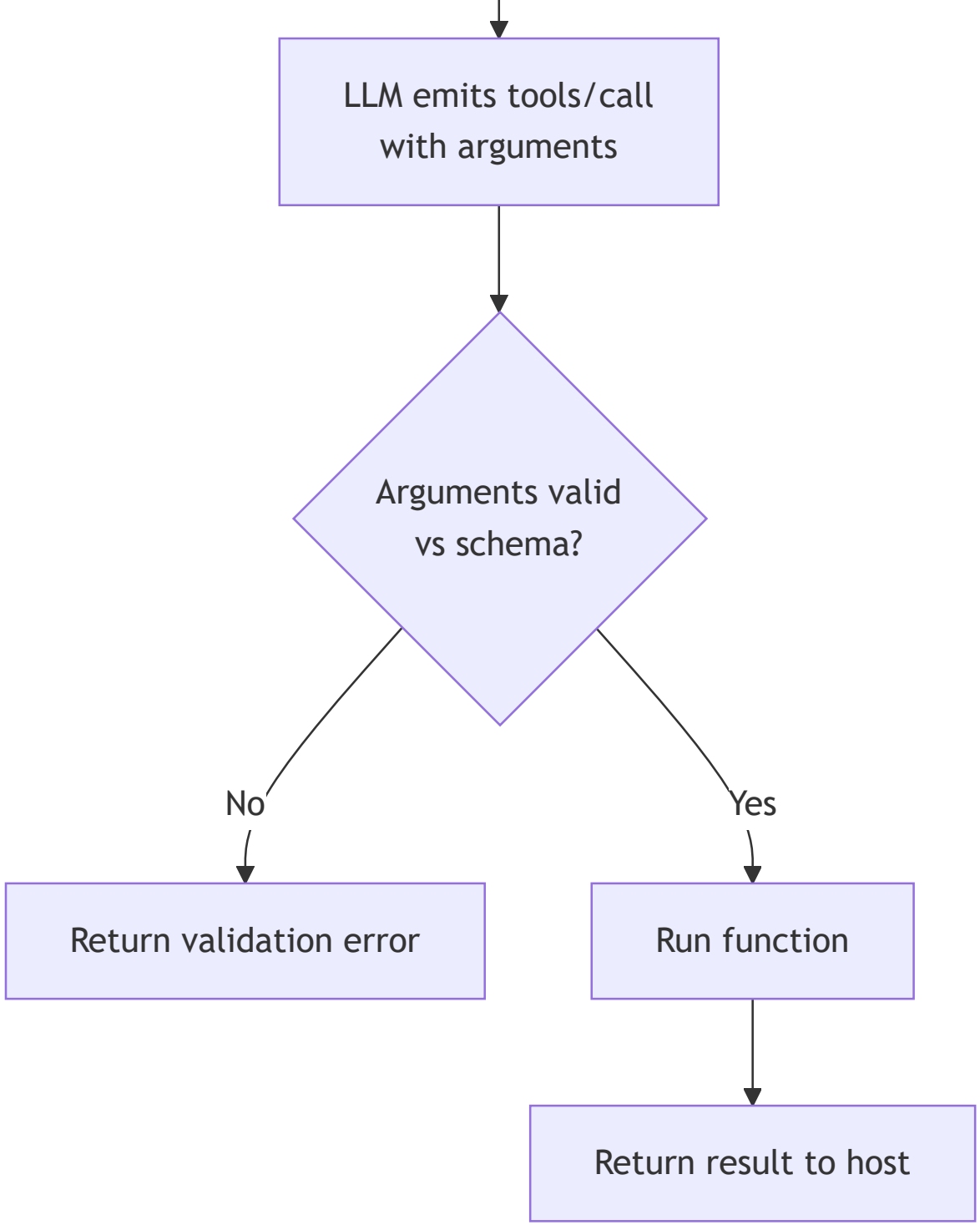


Figure 54.2: From decorated function to validated call: introspection at registration time produces a JSON Schema; at call time arguments are validated against it before the function runs.

This has several consequences worth internalizing:

- **Use type hints rigorously.** Untyped parameters default to `Any`, which produces a permissive schema and degrades the LLM's ability to call the tool correctly. Always type your parameters.
- **Use Pydantic for complex inputs.** When a parameter is a structured object, define it as a Pydantic `BaseModel` and use it as a type hint. FastMCP will produce the corresponding nested JSON Schema automatically.
- **Use `typing.Annotated` for descriptions.** A bare `int` becomes a JSON-Schema integer with no description. To attach per-field documentation that the LLM will see, use `Annotated[int, Field(description="...")]` or the FastMCP equivalent. This is the highest-leverage thing you can do to improve tool usability.
- **Default values matter.** Defaults become "not required" in the schema, which lets the model omit them. Default-rich functions are easier for models to call.

A more elaborate example:

```
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from fastmcp import FastMCP

mcp = FastMCP("weather")

class Location(BaseModel):
    city: str = Field(description="City name, e.g. 'San Francisco'")
    country: str = Field(description="Two-letter ISO country code, e.g. 'US'")

@mcp.tool
def get_forecast(
    location: Location,
    units: Annotated[Literal["metric", "imperial"], Field(description="Unit system")]
    days: Annotated[int, Field(ge=1, le=14, description="Forecast horizon")] = 3,
```

```
) -> str:
    """Get a multi-day weather forecast for a location."""
    # ... implementation ...
    return f"Forecast for {location.city}, {location.country} in {units} for {days} d
```

The resulting tool, as the LLM sees it, has a richly described nested schema with bounded integers and constrained literals. The model knows the country must be a two-letter code, units must be one of two strings, days is between 1 and 14. This kind of specificity dramatically reduces malformed tool calls.

54.2.4 Defining resources and prompts

Tools are the most common primitive, but FastMCP supports the other two MCP primitives with the same decorator-driven style.

Resources are exposed via `@mcp.resource(uri)`. The URI can be templated:

```
@mcp.resource("config://app/settings")
def app_settings() -> str:
    """Return the application's settings as a JSON string."""
    return json.dumps({"theme": "dark", "version": "1.0.0"})

@mcp.resource("user://{user_id}/profile")
def user_profile(user_id: str) -> str:
    """Return a user's profile by ID."""
    return f"Profile for user {user_id}"
```

The first resource has a static URI; the second has a template variable extracted from the URI path. When the host requests `user://42/profile`, FastMCP routes to `user_profile(user_id="42")`.

Prompts are exposed via `@mcp.prompt`:

```
@mcp.prompt
def review_pr(pr_number: int, focus: str = "correctness") -> str:
    """A prompt template for reviewing a PR with a particular focus."""
    return (
        f"Review pull request #{pr_number} with primary focus on {focus}. "
        "Identify bugs, suggest improvements, and call out any unclear naming."
    )
```

Hosts that support prompts (like Claude Desktop) typically expose these as slash-commands; the user types `/review_pr 123 correctness` and the prompt is templated and inserted as the user message.

54.2.5 Authentication and secrets

For local stdio servers, authentication is usually inherited: the server runs as the user, with whatever filesystem and environment access the user has. Secrets (API keys, OAuth tokens) are typically loaded from environment variables that the host passes to the subprocess at launch.

```
import os
from fastmcp import FastMCP

mcp = FastMCP("github-helper")
GITHUB_TOKEN = os.environ.get("GITHUB_TOKEN")
if not GITHUB_TOKEN:
    raise RuntimeError("Set GITHUB_TOKEN before running this server.")

@mcp.tool
def list_my_repos() -> list[str]:
    """List repos for the authenticated GitHub user."""
    # Use GITHUB_TOKEN to call the GitHub API.
    ...
```

The host's configuration file (e.g., Claude Desktop's `claude_desktop_config.json`) specifies environment variables passed to the server, which keeps secrets out of source code:

```
{
  "mcpServers": {
    "github-helper": {
      "command": "python",
      "args": ["/path/to/my_server.py"],
      "env": {"GITHUB_TOKEN": "ghp_..."}
    }
  }
}
```

For **remote (HTTP)** MCP servers, FastMCP supports OAuth 2.0 / OpenID Connect flows via middleware. The user's host, on first connection, walks them through an OAuth grant, and the resulting bearer token is presented on subsequent requests. This is the path for cloud-hosted MCP servers used by multiple users.

54.2.6 Error handling, logging, and observability

Production servers need to handle errors gracefully. FastMCP provides a `ToolError` exception that propagates a structured error back to the host (and onward to the model), so the model can see an explicit failure rather than a generic exception trace.

```
from fastmcp.exceptions import ToolError

@mcp.tool
def divide(a: float, b: float) -> float:
    """Divide a by b. Raises a clear error on division by zero."""
    if b == 0:
```

```
        raise ToolError("Cannot divide by zero. Try a non-zero divisor.")
    return a / b
```

The model will see `"Cannot divide by zero. Try a non-zero divisor."` as the tool result and, with reasonable prompting, recover by retrying with a sensible value. The recovery loop is what makes a teaching error message valuable: the model reads the structured failure and self-corrects.

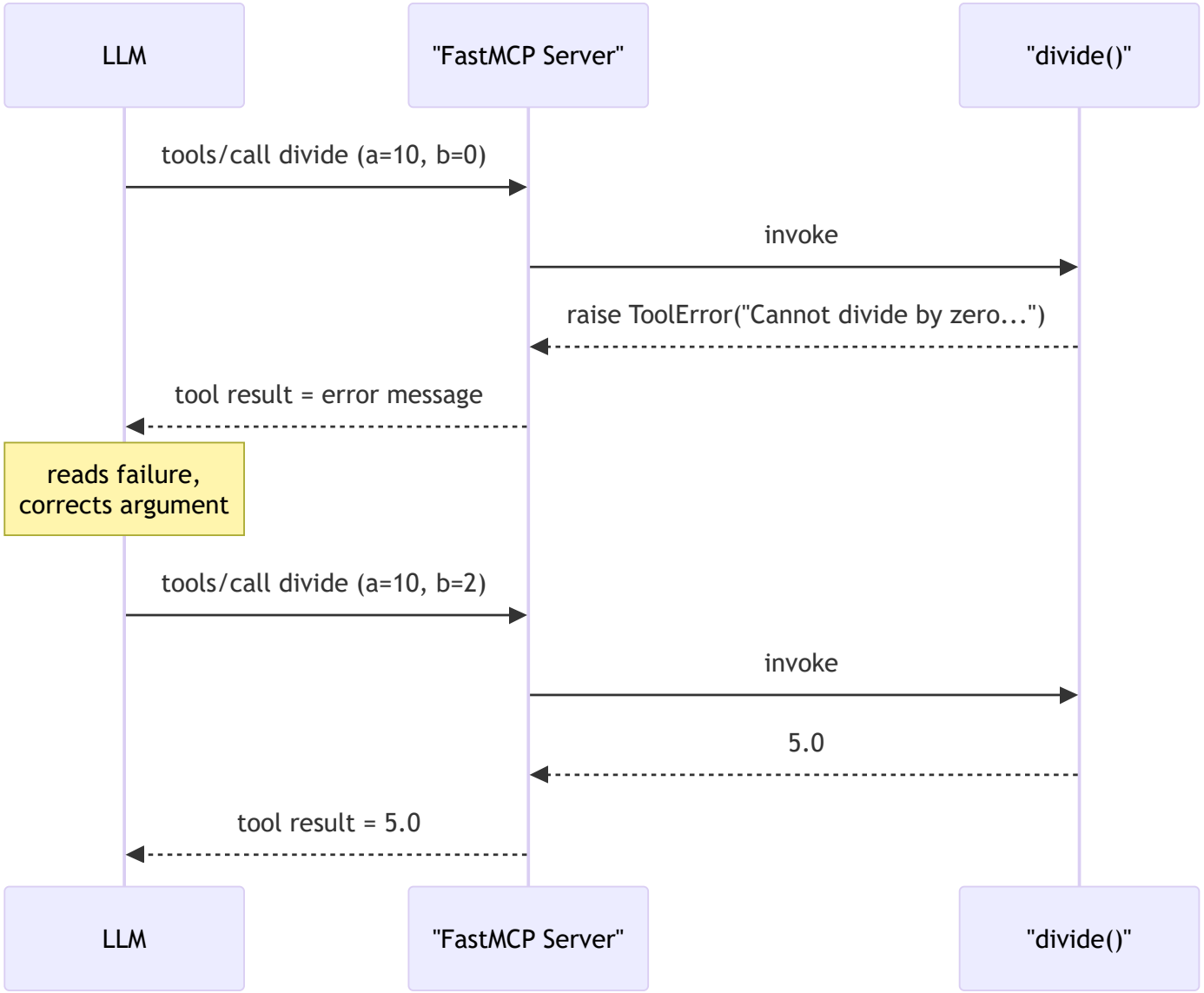


Figure 54.3: ToolError recovery loop: a structured error is returned to the model, which reads it and retries with corrected arguments instead of failing the task.

For logging, FastMCP provides a `ctx` parameter that any tool can accept to emit progress and log messages over the protocol:

```
from fastmcp import Context

@mcp.tool
async def slow_lookup(query: str, ctx: Context) -> str:
    """A slow lookup with progress reporting."""
    await ctx.info(f"Starting lookup for: {query}")
    await ctx.report_progress(0.5, "halfway done")
    # ... work ...
    await ctx.info("Lookup complete")
    return "result"
```

Hosts that support progress reporting (Claude Desktop, Cursor) display these to the user in real time. This is the killer feature for tools that take more than a couple of seconds; without it, the user has no idea whether the agent is hung or just slow.

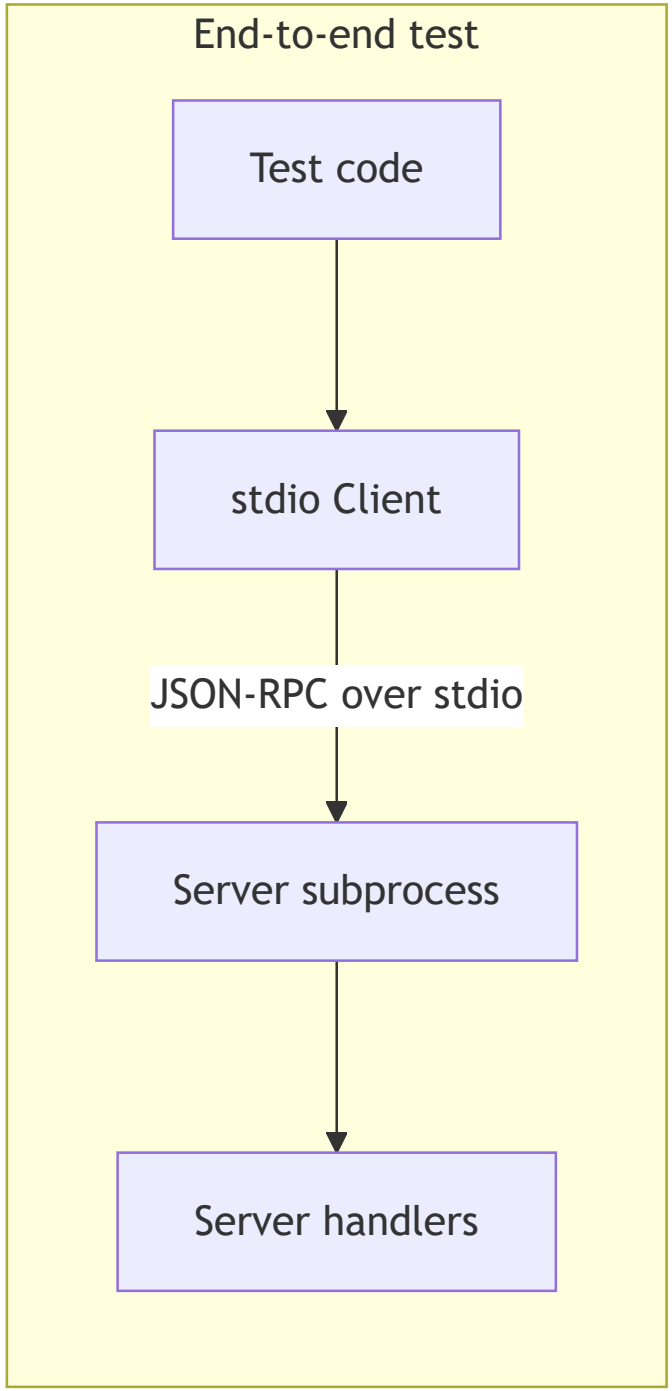
54.2.7 Testing servers locally

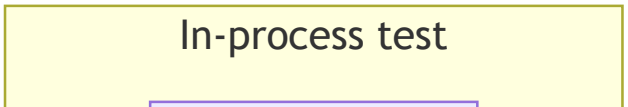
FastMCP includes a programmatic in-process client that lets you test a server without launching a subprocess. This is invaluable for unit tests:

```
import asyncio
from fastmcp import Client

async def test_add() -> None:
    async with Client(mcp) as client:  # 'mcp' is your FastMCP server object
        result = await client.call_tool("add", {"a": 2, "b": 3})
```

```
        assert result.content[0].text == "5"

asyncio.run(test_add())
```

The in-process client speaks the protocol directly to the server's handlers without any IPC, making tests fast and deterministic. The two test topologies differ only in whether a process and transport boundary sits between client and server.


End-to-end test
Test code
stdio Client
JSON-RPC over stdio
Server subprocess
Server handlers


In-process test

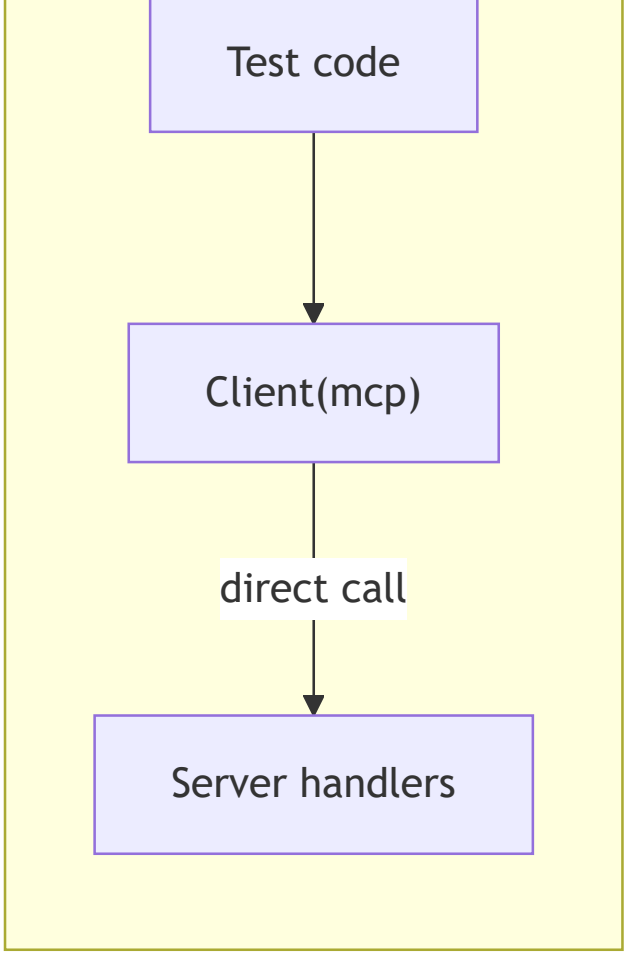


Figure 54.4: In-process testing calls server handlers directly with no IPC (fast, deterministic); end-to-end testing crosses a stdio subprocess boundary like a real host.

For a more end-to-end test, you can run the server as a subprocess and use the standard stdio client (as in the previous primer's example).

For interactive exploration, the **MCP Inspector** is a small web UI from the protocol authors that connects to any MCP server and lets you list tools, see their schemas, and invoke them with arbitrary arguments. This is the equivalent of Postman or `httpie` for MCP and you should run it whenever you write a new tool.

54.2.8 Packaging and distribution

A FastMCP server becomes shareable when you publish it. The two common patterns:

- **PyPI distribution** with a console script entry point. Add a `pyproject.toml` with a `[project.scripts]` entry mapping `my-server = "my_server.main:run"` and publish to PyPI. Users install with `pip install my-server` (or `uvx my-server`) and configure their host to launch the command.
- `uvx` **-direct** distribution. Because `uvx` can run a Python package directly from PyPI without prior install, server authors increasingly distribute by simply telling users to configure `command: "uvx"` and `args: ["my-server-name"]`. This is the smoothest UX for end users — they do not even need to install your server explicitly.

The `mcp.so` registry and the official `modelcontextprotocol/servers` repository serve as discovery channels. Once your server is published, listing it makes it findable.

54.2.9 Design principles for tools that LLMs can actually use

Building an MCP server that *exists* is easy. Building one that an LLM can *use well* is the harder, judgment-laden work. A few principles that matter in practice:

- **Granularity.** Avoid both ends of the spectrum. A single mega-tool (`do_anything(action: str, args: dict)`) makes the model guess at unstructured arguments and produces frequent malformed calls. Hundreds of micro-tools (`get_first_name`, `get_last_name`, `get_email`, …) bloat the tool catalog and confuse the model. Aim for tools that match the granularity of common user intents — `search_users`, `get_user`, `update_user`.
- **Naming.** Tool names are part of the prompt the model sees. Use clear, verb-prefixed names (`search_issues`, `create_pull_request`) that read like English. Avoid abbreviations.
- **Description quality.** Every tool needs a one-sentence description that says *what it does* and, if relevant, *when to use it versus a similar tool*. The description is your chance to disambiguate from related tools and steer the model.

- **Idempotency where possible.** Tools that can be safely retried are robust to model retries (which happen). For genuinely non-idempotent operations (sending emails, charging credit cards), require explicit confirmation tokens or rely on host-level confirmation prompts.
- **Structured returns.** Return text that is easy for the model to parse and act on. If the data is structured, return JSON or a clearly delimited block. Avoid HTML, ANSI escape codes, and other content that bloats context without helping the model.
- **Bounded outputs.** Truncate or paginate. A tool that returns 100,000 rows of a SQL result will blow your context budget in a single call. Cap output sizes and let the model paginate or refine.
- **Failure messages that teach.** When a tool fails, the error message is the model's only feedback. "Invalid parameter" is unhelpful; "City must be a non-empty string; got empty string. Example: 'San Francisco'." actually teaches the model how to recover.

These are the things that separate a tool you wrote in 20 minutes from a tool that an agent can drive correctly the first time, every time. Most of the post-launch work on a serious MCP server is iteration on descriptions and error messages.

54.2.10 When not to write an MCP server

A small but important point. If your tool is highly specific to one application (a single helper function only your one agent uses) and there is no plausible reuse value, MCP is overkill — just put it in your codebase. MCP makes sense when:

- The capability is reusable across multiple agents or hosts.
- The capability has a stable interface worth standardizing.
- You want to share it with others (or with a team).

Otherwise, in-process tool calling (a `@tool` decorator in LangChain, or a plain Python function) is simpler.

References

Anthropic. 2024. *Introducing the Model Context Protocol*. https://www.anthropic.com/news/model-context-protocol.

Lowin, Jeremiah. 2024. *FastMCP: The Fast, Pythonic Way to Build MCP Servers and Clients*. https://github.com/jlowin/fastmcp.

Model Context Protocol. 2024. *MCP Python SDK Documentation*. https://github.com/modelcontextprotocol/python-sdk.

Ramírez, Sebastián. 2018. *FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python*. https://fastapi.tiangolo.com/.